

Лекция 5

Пакеты и импорт сущностей в языке Scala

Актуальная задача при разработке больших программ – уменьшение зависимостей в коде

Один из инструментов уменьшения зависимостей – модульность (программа делится на фрагменты – модули, в которых чётко выделены части, относящиеся к интерфейсу и к реализации)

Современные языки программирования поддерживают на уровне синтаксиса тот или иной вариант модульности

Модуль в Scala называется *пакетом*. По умолчанию все объявления размещаются в безымянном пакете

Как и в Java, именование пакетов в Scala принято осуществлять по принципу «обратного доменного имени»:

```
ru.bmstu.iu9.schedule
```

В отличие от Java, язык Scala не соотносит пакеты со структурой каталогов файловой системы

Пакеты в Scala можно использовать как пространства имён в C#:

```
package имя_пакета {  
    ...  
    // Объявления  
    ...  
}
```

При этом в рамках одного файла можно часть объявлений разместить в одном пакете, а часть — в другом

Более того, пакеты могут вкладываться друг в друга

Доступ извне к сущностям, объявленным в пакете, осуществляется через *квалифицированные имена*:

```
имя_пакета.имя_сущности
```

Пример (вложенные пакеты и квалифицированные имена):

```
1 package ru.bmstu.iu9 {
2     package shapes {
3         class Point(val x: Int, val y: Int)
4         package rectangular {
5             class Rect(val p1: Point, val p2: Point)
6         }
7     }
8     object Constants {
9         val origin = new shapes.Point(0, 0)
10        val oneOne = new shapes.Point(1, 1)
11        val square = new shapes.rectangular.Rect(origin, oneOne)
12    }
13 }

15 object Test {
16     def main(args: Array[String]) {
17         val pt = new ru.bmstu.iu9.shapes.Point(10, 20)
18         println(pt.x, pt.y)
19     }
20 }
```

Правила обращения к сущностям, объявленным в пакетах:

1. обращение к сущности (классу, трейту, вложенному пакету) из пакета, в котором сущность объявлена, осуществляется по имени без префикса
2. сущности, объявленные «снаружи» пакета, т.е. в пакетах, в которые он вложен, доступны внутри пакета также без префикса
3. в остальных случаях, если не используется механизм импорта сущностей, для обращения к сущностям применяются их квалифицированные имена (в том числе – относительные)

В языке Scala существует альтернативный способ размещения объявлений в пакетах, действующий на уровне файла:

```
1 package ru.bmstu.iu9.shapes
2 package rectangular

4 class Rect(val p1: Point, val p2: Point)
```

Обратите внимание на то, что директивы `package` без фигурных скобок должны располагаться в начале файла до любых объявлений

Кроме того, отметим, что следующий код компилироваться не будет:

```
1 package ru.bmstu.iu9.shapes.rectangular

3 class Rect(val p1: Point, val p2: Point)
4    // error: not found: type Point
```

Т.е. для открытия имён из пакета `ru.bmstu.iu9.shapes` требуется явно обозначить вложенность пакетов двумя директивами `package`

В Scala сущности, принадлежащие именованному пакету, могут быть импортированы с помощью директивы `import`:

```
1 import ru.bmstu.iu9.shapes.Point

3 object Test {
4   def main(args: Array[String]) {
5     val pt = new Point(10, 20)
6     println(pt.x, pt.y)
7   }
8 }
```

Обращение к импортированным сущностям не требует указания квалифицированных имен

Если нужно импортировать все сущности из пакета, то в директиве `import` имя сущности заменяется на знак подчёркивания:

```
import ru.bmstu.iu9.shapes._
```

В отличие от Java, директиву `import` можно использовать в любом месте файла. При этом область её действия будет ограничена блоком, в который она входит. Например,

```
1 object Test {  
2   import ru.bmstu.iu9.shapes._  
3   def main(args: Array[String]) {  
4     val pt = new Point(10, 20)  
5     println(pt.x, pt.y)  
6   }  
7 }
```

А можно вот так:

```
1 object Test {  
2   def main(args: Array[String]) {  
3     import ru.bmstu.iu9.shapes._  
4     val pt = new Point(10, 20)  
5     println(pt.x, pt.y)  
6   }  
7 }
```


Возможности директивы `import` выходят за рамки системы пакетов. Её можно использовать для импорта членов любого объекта:

```
1 object Test {
2   import ru.bmstu.iu9.shapes._

4   def printPoint(p: Point) {
5     import p._
6     println(x, y)
7   }

9   def main(args: Array[String]) {
10     val pt = new Point(10, 20)
11     printPoint(pt)
12   }
13 }
```

В директиве `import` можно использовать так называемый *селектор*, разрешающий явно указать список импортируемых сущностей и, кроме того, переименовать или спрятать некоторые сущности

Синтаксис селекторов:

`import ru.bmstu.iu9.Constants.{origin, oneOne}` — импортировать только сущности `origin` и `oneOne`

`import ru.bmstu.iu9.Constants.{origin=>zeroZero, square}` — импортировать только сущности `origin` и `square`, переименовав `origin` в `zeroZero`

`import ru.bmstu.iu9.Constants.{origin=>zeroZero, _}` — импортировать все сущности из `Constants`, переименовав `origin` в `zeroZero`

`import ru.bmstu.iu9.Constants.{square=>_, _}` — импортировать все сущности из `Constants`, спрятав сущность `square`

Пример использования селекторов:

```
1 object Test {
2   import ru.bmstu.iu9._
3   import Constants.{oneOne=>_, _}

5   def printRect(r: shapes.rectangular.Rect) {
6     import r.p1.{x, y}
7     import r.p2.{x=>x2, y=>y2}
8     println(x, y, x2, y2)
9   }

11  def main(args: Array[String]) {
12    printRect(square)
13  }
14 }
```

В языке Scala существует возможность объявлять в пакете не только классы, трейты и вложенные пакеты, но и поля и методы. Для этого предусмотрены так называемые *пакетные объекты*

Объявление пакетного объекта:

```
package object имяПакета {  
    ...  
}
```

Если для некоторого пакета объявлен пакетный объект, то из этого пакета можно импортировать поля и методы этого объекта

Пакетные объекты широко используются как хранилища констант, псевдонимов типов и методов, реализующих неявные преобразования типов

Пример объявления пакетного объекта:

```
1 package ru.bmstu {
2     package iu9 {
3         class Point(val x: Int, val y: Int)
4     }
5     package object iu9 {
6         val origin = new Point(0, 0)
7         def printPoint(p: Point) {
8             import p._
9             println(x, y)
10        }
11    }
12 }

14 object Test {
15     def main(args: Array[String]) {
16         import ru.bmstu.iu9._
17         printPoint(origin)
18     }
19 }
```

Компилятор Scala неявно импортирует все сущности из трёх специальных пакетов:

```
import java.lang._  
import scala._  
import Predef._
```

Пакет `java.lang` принадлежит стандартной библиотеке языка Java

Пакет `scala` содержит стандартные классы языка Scala (например, класс `List`)

Объект `Predef` содержит определения типов, методов и неявных преобразований, часто используемых в программах, написанных на Scala